

CS 5135 - Project

Mark Farley

Spring 2026

Question 1

A natural formulation for the problem can be defined as:

Data:

- r_i : number of proctors needed for exam i , $\forall i \in \{1, \dots, m\}$
- R : maximum number of proctors that can be deployed in total across all exams running simultaneously
- $G = (V, E)$: conflict graph where $V = \{1, \dots, m\}$ is the set of exams and an edge $(i, i') \in E$ indicates that exams i and i' have at least one student in common

Decision Variables:

•

$$x_{ij} = \begin{cases} 1, & \text{if exam } i \text{ is scheduled during timeslot } j \\ 0, & \text{otherwise} \end{cases} \quad \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, n\}$$

•

$$y_j = \begin{cases} 1, & \text{if we use timeslot } j \\ 0, & \text{otherwise} \end{cases} \quad \forall j \in \{1, \dots, n\}$$

Optimization Model:

$$\begin{aligned} \min \quad & \sum_{j=1}^n y_j \\ \text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1 && \forall i \in \{1, \dots, m\} \\ & \sum_{i=1}^m r_i x_{ij} \leq R y_j && \forall j \in \{1, \dots, n\} \\ & x_{ij} + x_{i'j} \leq 1 && \forall (i, i') \in E, \forall j \in \{1, \dots, n\} \\ & x_{ij}, y_j \in \{0, 1\} && \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, n\} \end{aligned}$$

As we can quite easily see, this natural formulation aims to minimize the total number of timeslots used. The first set of constraints ensure that each exam is scheduled to exactly one timeslot. The second set of constraints ensure that if a timeslot is in use, the total number of proctors deployed during that timeslot does not exceed the maximum number of proctors that can be deployed simultaneously; else, no proctors will be deployed (meaning no exams will take place). The third set of constraints ensure that we never schedule two exams that have at least one student in common in the same timeslot. Finally, we make sure that our decision variables can only take binary values, due to how they are defined.

In order to strengthen the constraints of our natural formulation, we can exploit the notion of clique in our conflict graph, G . Let K' denote the collection of all maximal cliques of G . Keeping the data and decision variables the same, we can then rewrite the optimization model as follows:

$$\begin{aligned}
\min \quad & \sum_{j=1}^n y_j \\
\text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1 && \forall i \in \{1, \dots, m\} \\
& \sum_{i=1}^m r_i x_{ij} \leq R y_j && \forall j \in \{1, \dots, n\} \\
& x_{ij} + x_{i'j} \leq 1 && \forall (i, i') \in E, \forall j \in \{1, \dots, n\} \\
& \sum_{i \in K} x_{ij} \leq 1 && \forall K \in K', \forall j \in \{1, \dots, n\} \\
& x_{ij}, y_j \in \{0, 1\} && \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, n\}
\end{aligned}$$

Each maximal clique denotes a set of exams that at least one student has in common. Thus, we are able to replace our old third set of constraints with a new set of constraints, utilizing maximal cliques instead of just cliques of size two. In Python, finding maximal cliques is easy using the NetworkX package. You can simply convert a list of edges into a graph and call `find_cliques()`, passing that graph as an argument. However, we now have exponentially many constraints. We have created a “strong” model.

An alternative is to create a more “intermediate” model that still utilizes some maximal cliques, but has no more than the n constraints our natural formulation had. Our optimization model would instead look like this:

$$\begin{aligned}
\min \quad & \sum_{j=1}^n y_j \\
\text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1 & \forall i \in \{1, \dots, m\} \\
& \sum_{i=1}^m r_i x_{ij} \leq R y_j & \forall j \in \{1, \dots, n\} \\
& x_{ij} + x_{i'j} \leq 1 & \forall (i, i') \in E, \forall j \in \{1, \dots, n\} \\
& \sum_{i \in K} x_{ij} \leq 1 & \text{for each } (i, i') \in E \text{ and for some } K \in K' \\
& & \text{such that } (i, i') \in K, \forall j \in \{1, \dots, n\} \\
& x_{ij}, y_j \in \{0, 1\} & \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, n\}
\end{aligned}$$

Essentially, we will be replacing each constraint in our third set of constraints in our natural formation with a maximal clique containing the vertices in each constraint, possibly removing some redundant constraints as well. In Python, this was done by calling `find_cliques()`, passing the graph and each edge as an argument, and arbitrarily selecting the first clique returned for each set of edges.

Question 2

An extended formulation for the problem can be defined as:

Data:

- r_i : number of proctors needed for exam i , $\forall i \in \{1, \dots, m\}$
- R : maximum number of proctors that can be deployed in total across all exams running simultaneously
- $G = (V, E)$: conflict graph where $V = \{1, \dots, m\}$ is the set of exams and an edge $(i, i') \in E$ indicates that exams i and i' have at least one student in common
- S' : collection of every feasible exam grouping where $S \subseteq \{1, \dots, m\} \in S'$ is a feasible exam grouping if and only if $\sum_{i \in S} r_i \leq R$ and S is a stable set in G

Decision Variables:

•

$$x_S = \begin{cases} 1, & \text{if in a solution the exams in } S \text{ happen during the same timeslot} \\ 0, & \text{otherwise} \end{cases}$$

Optimization Model:

$$\begin{aligned}
\min \quad & \sum_{S \in S'} x_S \\
\text{s.t.} \quad & \sum_{S \in S': i \in S} x_S = 1 & \forall i \in \{1, \dots, m\} \\
& x_S \in \{0, 1\} & \forall S \in S'
\end{aligned}$$

The model above requires us to compute every single combination of exams that can be taken at the same time, significantly reducing the calculations that solver has to make. The solver just has to pick a combination of feasible exam groupings where each exam only happens once. However, we now have an exponential number of variables.

The model described above is known as the set partitioning extended formulation. In the set partitioning formulation, we are choosing feasible exam groupings from the collection of every feasible exam grouping so that each exam is essentially assigned to a single timeslot. However, with some modifications, we can also create the set covering extended formulation. Suppose instead we replaced S' with the collection S'' , which would be a collection of every maximal feasible exam grouping instead, meaning that if we tried to schedule any additional exam in any of the feasible exam groupings, we would exceed R . We would then be able to define the set covering extended formulation as:

Data:

- r_i : number of proctors needed for exam i , $\forall i \in \{1, \dots, m\}$
- R : maximum number of proctors that can be deployed in total across all exams running simultaneously
- $G = (V, E)$: conflict graph where $V = \{1, \dots, m\}$ is the set of exams and an edge $(i, i') \in E$ indicates that exams i and i' have at least one student in common
- S'' : collection of every maximal feasible exam grouping where $S \subseteq \{1, \dots, m\} \in S''$ is a maximal feasible exam grouping if and only if $\sum_{i \in S} r_i \leq R$, S is a stable set in G , and scheduling any additional exam in S will cause the proctor requirement to exceed R

Decision Variables:

•

$$x_S = \begin{cases} 1, & \text{if in a solution the exams in } S \text{ happen during the same timeslot} \\ 0, & \text{otherwise} \end{cases}$$

Optimization Model:

$$\begin{aligned}
\min \quad & \sum_{S \in S''} x_S \\
\text{s.t.} \quad & \sum_{S \in S'': i \in S} x_S \geq 1 & \forall i \in \{1, \dots, m\} \\
& x_S \in \{0, 1\} & \forall S \in S''
\end{aligned}$$

Both of these extended formulations correspond to a solution in the other extended formulation with the same objective value. We can convert a set partitioning solution to a set covering solution by determining a maximal subset that each of our exam groupings are already a part of and setting the decision variable corresponding to that maximal subset to 1. We can also convert a set covering solution to a set partitioning solution because the solution to the set covering formulation will be a minimal solution (no exam grouping is entirely composed of exams that are all also a subset of a different exam grouping). Thus, we can go through each maximal subset and put any exams that we have not converted into a set partitioning solution into a subset and select that subset as a part of our set partitioning solution.

The set partitioning formulation can be seen as a more direct (and possibly “intuitive”) formulation. It essentially directly creates the exam schedule, where every chosen subset is a set of exams that will be taken at the same time.

The set covering formulation does require a little bit of extra work to translate into a solution that can be used (and possibly a little bit less “intuitive,” but not by much). We can schedule an exam to happen with any of the subsets it is a part of. If we choose an exam to be scheduled with a particular subset, we obviously will not schedule it with the other subsets it is a part of.

While the set covering formulation does require a bit of extra work, the formulation is still better than the set partitioning formulation because the number of variables is smaller (though still exponential), both formulations have the same LP relaxation value (choosing set partitioning does not help us with our LP bound), and LPs with inequalities are generally easier to solve with LPs than equations.

As the problem did not define S to be a maximal feasible grouping, I chose to implement the set partitioning formulation.

Question 3

Dataset 1

The solver data for each instance of dataset 1 is as follows:

Natural Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	8	6.32	21	7.259	Yes
2	9	7.32	7026	21.272	Yes
3	9	7.88	339	14.611	Yes
4	14	12.84	14259	120.025	No
5	13	12.40	1	17.487	Yes
6	14	12.88	2401	120.023	No
7	13	12.68	5153	114.461	Yes
8	16	15.36	1	3.096	Yes
9	15	14.16	1	6.535	Yes
10	16	14.68	3589	120.051	No

Intermediate Clique-Strengthened Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	8	6.32	4886	19.178	Yes
2	9	7.32	11586	39.815	Yes
3	9	7.88	6249	25.638	Yes
4	14	12.84	3015	120.068	No
5	13	12.40	282	28.002	Yes
6	14	12.88	2384	120.118	No
7	14	12.68	2203	120.171	No
8	16	15.36	2312	102.222	Yes
9	16	14.16	1414	120.100	No
10	16	14.68	858	120.109	No

Strong Clique-Strengthened Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	8	6.32	4120	15.729	Yes
2	9	7.32	8679	120.008	No
3	9	7.88	11598	120.059	No
4	14	12.84	1295	120.165	No
5	13	12.40	634	89.713	Yes
6	14	12.88	188	120.262	No
7	15	12.68	141	120.179	No
8	17	15.36	21	120.261	No
9	16	14.16	33	120.290	No
10	16	14.68	1	120.164	No

Extended Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	8	7.18	1	0.040	Yes
2	9	7.92	1	0.097	Yes
3	9	8.12	1	0.073	Yes
4	13	12.84	1	1.681	Yes
5	13	12.40	1	0.312	Yes
6	13	12.88	343	2.503	Yes
7	13	12.68	1	2.683	Yes
8	16	15.36	1	0.524	Yes
9	15	14.16	1	0.698	Yes
10	15	14.68	1	0.605	Yes

Clique strengthening does not seem to improve our LP bound over the natural formulation. I would have expected it to affect at least one of the bounds but that does not seem to be the case. I believe that at least part of the reason why this may be the case is the pre-processing done by the solver. The presolve phase determines the clique table or conflict graph as well, which is most likely already doing some of the work I did when implementing the clique-strengthening formulations.

The extended formulation, on the other hand, does seem to improve our LP bound over our other formulations for our smaller instances (1-3). I would attribute the reasoning to be similar to what I have just mentioned. I believe that the presolve phase already does good work in determining an LP bound. However, as our formulation is much more different from just performing clique strengthening, we are able to get a higher LP bound for our smaller instances due to their structure. The exact reasoning behind this, however, I am not quite sure.

We can quite easily see that formulations with equal LP relaxation values do not always explore a similar number of nodes, nor require similar computing time. Our natural and clique-strengthened formulations all had equal LP relaxation values but, in general, the clique-strengthened formulations seem to need to explore more branch-and-bound nodes while also requiring significantly more time. If we only look at the instances that were solved to optimality, we can see that our clique-strengthened formulations explored significantly more nodes when compared to our natural formulation. In addition, while the data present here is not enough to formulate an actual conclusion, I would speculate that our strong clique-strengthened formulation would generally need to explore significantly more nodes than our intermediate clique-strengthened formulation. Our clique-strengthened formulations (especially the strong formulation) also generally seem to require significantly more time to solve the instances. The natural formulation was able to solve 7 instances within the time limit while the intermediate formulation was only able to solve 5 and the strong formulation was only able to solve 2. In addition, the objective solution we reached within the time limit also seems to get worse as we make our clique strengthening stronger. In addition, as our instances grow in size, the clique-strengthened formulations seem to require more time at each node (as shown by exploring fewer nodes while not reaching the optimal solution within the time limit).

The extended formulation really exemplifies how formulations with equal LP relaxation values can have an extreme difference in performance. The extended formulation was able to solve every instance in less than 3 seconds, with most requiring less than 1 second. Meanwhile, the fastest solve outside of this formulation was on instance 8 in the natural formulation, with a time of just over 3 seconds. This includes instances that previously could not be solved within the 120 second time limit. In addition, it only had to explore 1 node in 9 of the instances (the only exception is in instance 6, where it had to explore 343 nodes).

My data suggests that LP bound quality cannot be a reliable predictor of branch-and-bound performance on its own. While a better bound can help us know if we have reached the optimal solution earlier, depending on our formulation, branch-and-bound can have wildly different performance. Our extended formulation only required 1 node (except for one instance) and less than 3 seconds to solve instances that had the same LP bound in other formulations. Meanwhile, our clique-strengthened formulations generally required significantly more nodes and time to solve when compared to our natural formulation, despite also having the same LP bound.

As instance sizes grow, computational effort does seem to evolve differently for each formulation. For our natural formulation, we are still able to handle having to explore thousands of nodes quite well, and in some cases we are able to reach the optimal solution with just 1 node. In addition, for many of the instances, we are able to solve them within the time limit. Instances 8 and 9 took around 3.096 and 6.535 seconds respectively, showing that even with these larger formations, our model can sometimes still solve them quite quickly. Meanwhile, our clique-strengthened formulations show a

different story. Generally speaking, as our instances grew in size, the number of nodes we were able to explore within the time limit decreased dramatically, suggesting that more time is being spent at each node. In addition, our solutions seem to take much longer to reach, as demonstrated by the longer solve times compared to the natural formulation and the amount of instances we were unable to solve in time. The extended formulation also shows a clear difference in evolution, generally requiring only 1 node to solve each of the instances and solving every instance within 3 seconds (and most instances within 1).

The rate of growth clearly differs across each of the formulations. The amount of nodes for the natural formulation seems somewhat unpredictable depending on the instance (some require 1, some thousands, etc.) but the solve time does not see massive growth (we do not solve some instances but we solve some of the larger instances, suggesting that it is more dependent on the structure of the instance itself rather than just the size of it). The clique-strengthened formulations seem to require more nodes to solve the instances (though it is hard to predict with the limited solves we have), requires significantly more time to solve the instance, and the time spent at each node seems to grow quite heavily as our instances increase in size. Finally, the extended formulation does not really seem to grow much, if at all, in terms of nodes explored and solve time as our instance sizes grow.

Dataset 2

The solver data for each instance of dataset 2 is as follows:

Natural Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	17	16.32	175	20.873	Yes
2	17	16.08	1	5.035	Yes
3	17	16.28	255	37.577	Yes
4	18	16.76	16167	600.066	No
5	17	16.60	2410	253.295	Yes
6	18	17.76	2190	134.732	Yes
7	18	16.88	18386	600.041	No
8	17	16.84	6053	372.528	Yes
9	19	18.00	5715	600.104	No

Intermediate Clique-Strengthened Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	17	16.32	308	70.174	Yes
2	17	16.08	1	45.328	Yes
3	17	16.28	1	37.581	Yes
4	18	16.76	2229	600.220	No
5	17	16.60	1077	212.618	Yes
6	18	17.76	1975	537.799	Yes
7	18	16.88	2122	600.119	No
8	18	16.84	2905	600.185	No
9	19	18.00	1602	600.231	No

Strong Clique-Strengthened Formulation

Instance	Objective value reached	LP relaxation value	Branch-and-Bound Nodes Explored	Wall-Clock Solve Time (seconds)	Solved to Optimality in Time Limit?
1	18	16.32	113	601.095	No
2	17	16.08	1	317.047	Yes
3	18	16.28	31	601.542	No
4	18	16.76	20	600.857	No
5	19	16.60	8	600.750	No
6	19	17.76	33	600.630	No
7	18	16.88	128	600.648	No
8	19	16.84	36	600.795	No
9	19	18.00	1	601.213	No

As we can quite easily see, as the instances grow in size, the natural formulation and clique-strengthened formulations trend towards requiring significantly more time. For the natural formulation, we were only able to solve the 3 smallest formulations within the old 120 second time limit. The same thing happens with the intermediate clique-strengthened formulation. However, with the strong clique-strengthened formulation, we were unable to solve 9 of the instances within the new 600 second time limit, and the 1 instance that was solved was not solved within 120 seconds. Overall, we can see that the natural formulation is able to still solve most of these instances within a reasonable time, though it still does take significantly more time than before. As the instance sizes grow even more, I suspect that this gap will become significantly more apparent. However, the clique-strengthened formulations (especially the strong one) are already beginning to more clearly show their computational limits. The strong formulation already seems to be becoming impractical in pretty much all cases while the intermediate formulation is showing more impracticality as well.

The extended formulation is not applicable to these instances due to the number of variables required being exponential, caused by computing every single combination of exams that can be taken at the same time (stable set problem). At this point, we are starting to reach instance sizes that could be difficult for a solver to solve. While a “reasonable” value for R does help cut down on the number of variables, instances can be constructed where we do keep an exponential number of variables, causing the solver to take significantly more time to solve the problem.